RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Data Science

V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 7: Model Planning and Building ? Clustering

Name: muthukutti R | Register No: 241801175 | Date: 10/6/26

Dataset: Mall Customer Segmentation Data (Kaggle) **Tools:** Python, Pandas, scikit-learn, Matplotlib, Seaborn

Aim

To group mall customers based on their Annual Income and Spending Score using the K-Means clustering algorithm, find the optimal number of clusters using the Elbow Method, visualise and analyse the customer segments, and recommend suitable marketing strategies for each segment.

Task 1: Load the Mall Customer Segmentation dataset

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

import warnings
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')

df = pd.read_csv('Mall_Customers.csv')
print("Dataset loaded successfully")
```

Out [1]:

```
Dataset loaded successfully
```

Task 2: Explore the dataset

First and last five records

In [2]:

```
df.head()
```

Out [2]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

5 rows × 5 columns

In [3]:

```
df.tail()
```

Out [3]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

5 rows × 5 columns

Shape and data types

In [4]:

```
print("Shape of the dataset:", df.shape)
print("\nData types:")
print(df.dtypes)
```

Out [4]:

```
Shape of the dataset: (200, 5)

Data types:
CustomerID                int64
Gender                      str
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

Missing values and duplicate records

In [5]:

```
print("Missing values in each column:")
print(df.isnull().sum())
print("\nTotal missing values      :", df.isnull().sum().sum())
print("Duplicate records         :", df.duplicated().sum())
print("Duplicate CustomerID      :", df['CustomerID'].duplicated().sum())
```

Out [5]:

```
Missing values in each column:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Total missing values      : 0
Duplicate records         : 0
Duplicate CustomerID      : 0
```

Summary statistics

In [6]:

```
df.describe().round(2)
```

Out [6]:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.00	200.00	200.00	200.00
mean	100.50	38.85	60.56	50.20
std	57.88	13.97	26.26	25.82
min	1.00	18.00	15.00	1.00
25%	50.75	28.75	41.50	34.75
50%	100.50	36.00	61.50	50.00
75%	150.25	49.00	78.00	73.00
max	200.00	70.00	137.00	99.00

8 rows × 4 columns

In [7]:

```
print(df['Gender'].value_counts())
```

Out [7]:

```
Gender
Female    112
Male       88
Name: count, dtype: int64
```

> **Observation:** The dataset has 200 customers and 5 columns with no missing values or duplicate records. Customers are 18 to 70 years old, their annual income ranges from 15 to 137 k$ and their spending score from 1 to 99. There are 112 female and 88 male customers.

Task 3: Select the features for clustering

In [8]:

```
features = ['Annual Income (k$)', 'Spending Score (1-100)']
X = df[features]

print("Selected features:", features)
print("Shape of X:", X.shape)
X.head()
```

Out [8]:

```
Selected features: ['Annual Income (k$)', 'Spending Score (1-100)']
Shape of X: (200, 2)
```

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

5 rows × 2 columns

Task 4: Standardise the selected features

In [9]:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(X_scaled, columns=['Income (scaled)', 'Spending (scaled)'])
print("Mean after scaling             :", np.abs(X_scaled.mean(axis=0)).round(4))
print("Standard deviation after scaling:", X_scaled.std(axis=0).round(4))
X_scaled_df.head()
```

Out [9]:

```
Mean after scaling             : [0. 0.]
Standard deviation after scaling: [1. 1.]
```

	Income (scaled)	Spending (scaled)
0	-1.738999	-0.434801
1	-1.738999	1.195704
2	-1.700830	-1.715913
3	-1.700830	1.040418
4	-1.662660	-0.395980

5 rows × 2 columns

> **Observation:** Income (in k$) and spending score (1–100) are on different scales. After standardisation both features have a mean of 0 and a standard deviation of 1, so both contribute equally to the distance calculations in K-Means.

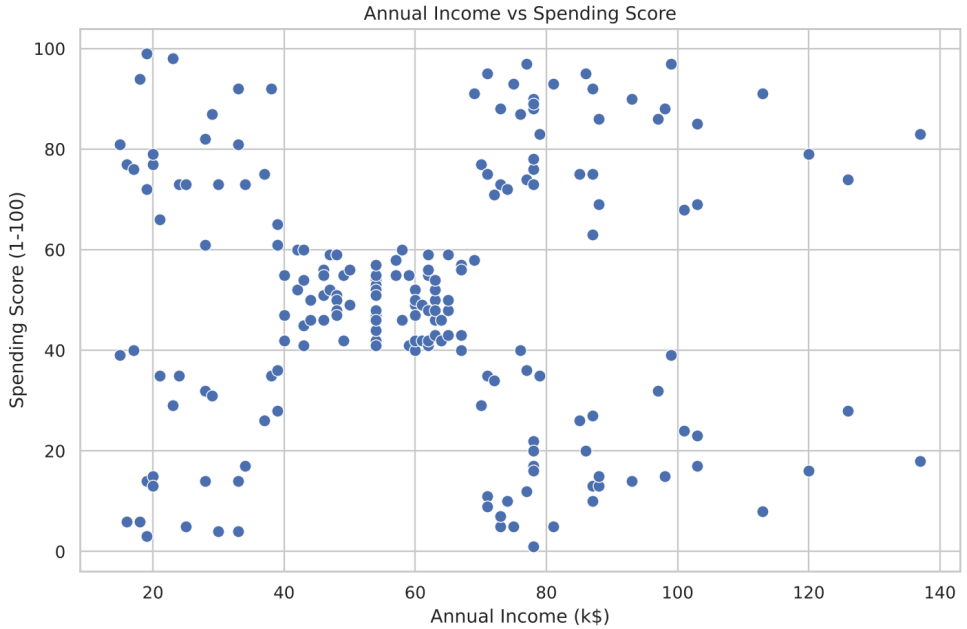
Task 5: Visualise the selected features

Scatter Plot

In [10]:

```
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)',
                color='#4C72B0', s=60, edgecolor='white')
plt.title('Annual Income vs Spending Score')
plt.tight_layout()
plt.show()
```

Out [10]:

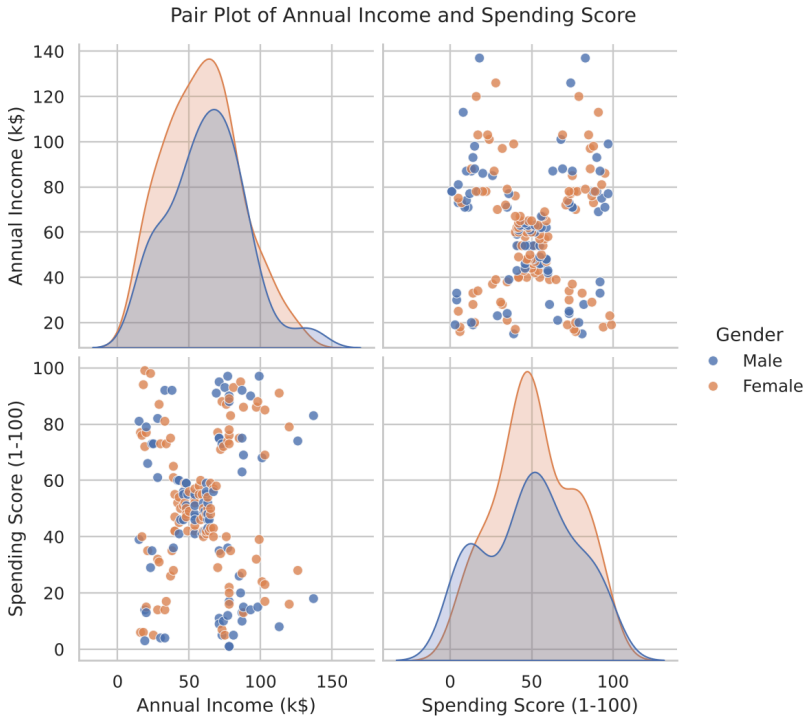


Pair Plot

In [11]:

```
sns.pairplot(df[features + ['Gender']], hue='Gender', palette={'Male': '#4C72B0', 'Female':
'#DD8452'},
             diag_kind='kde', height=3.2, plot_kws={'s': 40, 'alpha': 0.8})
plt.suptitle('Pair Plot of Annual Income and Spending Score', y=1.03, fontsize=13)
plt.show()
```

Out [11]:



> **Observation:** The scatter plot shows about five natural groups of customers: one large group in the middle, and four groups in the corners (low/high income combined with low/high spending). The pair plot shows that male and female customers follow a similar pattern, so gender is not needed to separate the groups.

Model Planning and Building

Task 6: Find the optimal number of clusters using the Elbow Method

The Elbow Method plots the WCSS (Within-Cluster Sum of Squares, called *inertia* in scikit-learn) for different values of K. The optimal K is at the 'elbow', where adding more clusters stops reducing the WCSS by much.

In [12]:

```
wcss = []
silhouette = []
k_values = range(1, 11)
for k in k_values:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
    silhouette.append(silhouette_score(X_scaled, km.labels_) if k > 1 else np.nan)

elbow_table = pd.DataFrame({'K': list(k_values), 'WCSS': np.round(wcss, 2),
                            'Drop in WCSS (%)': (-pd.Series(wcss).pct_change() * 100).round(1),
                            'Silhouette Score': np.round(silhouette, 4)}).set_index('K')
elbow_table
```

Out [12]:

K	WCSS	Drop in WCSS (%)	Silhouette Score
1	400.00	NaN	NaN
2	269.69	32.6	0.3213
3	157.70	41.5	0.4666
4	108.92	30.9	0.4939
5	65.57	39.8	0.5547
6	55.06	16.0	0.5399
7	44.86	18.5	0.5281
8	37.23	17.0	0.4552
9	32.39	13.0	0.4571
10	29.98	7.4	0.4432

10 rows × 3 columns

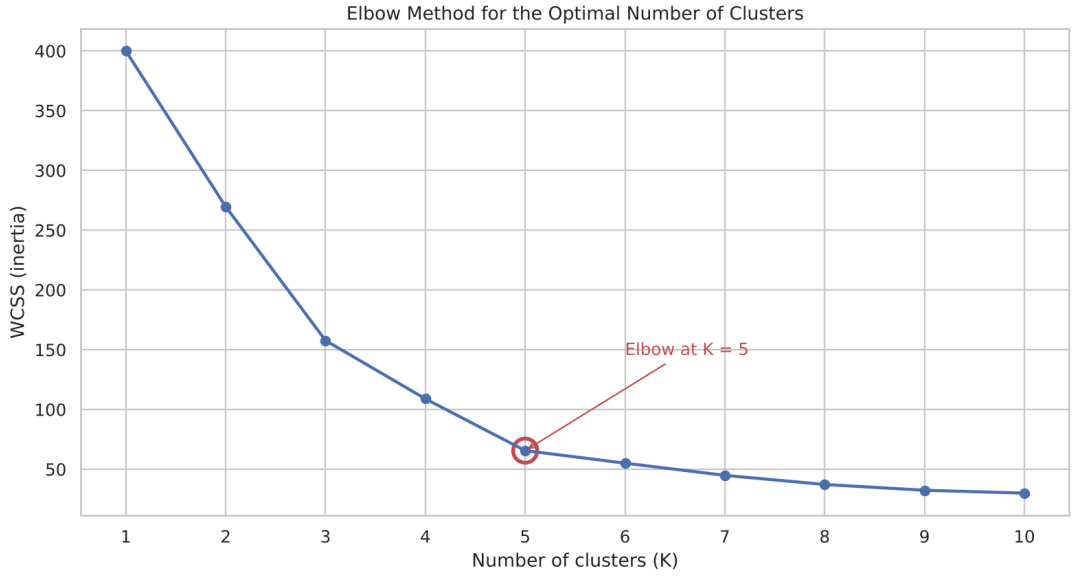
In [13]:

```
optimal_k = 5

plt.figure(figsize=(10, 5.5))
plt.plot(list(k_values), wcss, marker='o', color='#4C72B0', linewidth=2)
plt.scatter(optimal_k, wcss[optimal_k - 1], s=250, facecolors='none', edgecolors='#C44E52',
linewidths=2.5)
plt.annotate(f'Elbow at K = {optimal_k}', xy=(optimal_k, wcss[optimal_k - 1]),
             xytext=(optimal_k + 1, wcss[optimal_k - 1] + 80), fontsize=11, color='#C44E52',
             arrowprops=dict(arrowstyle='->', color='#C44E52'))
plt.title('Elbow Method for the Optimal Number of Clusters')
plt.xlabel('Number of clusters (K)')
plt.ylabel('WCSS (inertia)')
plt.xticks(list(k_values))
plt.tight_layout()
plt.show()

print("Optimal number of clusters (K):", optimal_k)
```

Out [13]:




```
Optimal number of clusters (K): 5
```

> **Observation:** The WCSS falls sharply up to K = 5 (a 39.8% drop from K = 4 to K = 5) and then decreases only slowly (16% or less for each extra cluster), so the elbow is at **K = 5**. The silhouette score is also highest at K = 5 (0.5547), which confirms this choice.

Task 7: Train the K-Means model and assign cluster labels

In [14]:

```
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', n_init=10, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

print("K-Means model trained with K =", optimal_k)
print("Final WCSS       :", round(kmeans.inertia_, 2))
print("Silhouette score :", round(silhouette_score(X_scaled, df['Cluster']), 4))
df.head(10)
```

Out [14]:

```
K-Means model trained with K = 5
Final WCSS       : 65.57
Silhouette score : 0.5547
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	2
8	9	Male	64	19	3	4
9	10	Female	30	19	72	2

10 rows × 6 columns

In [15]:

```
df.to_csv('Mall_Customers_Clustered.csv', index=False)
print("Saved: Mall_Customers_Clustered.csv")

saved = pd.read_csv('Mall_Customers_Clustered.csv')
print("Shape of the saved file:", saved.shape)
print("Columns:", saved.columns.tolist())
```

Out [15]:

```
Saved: Mall_Customers_Clustered.csv
Shape of the saved file: (200, 6)
Columns: ['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)', 'Cluster']
```

Cluster Visualisation

Task 8: Customer clusters with their centroids

In [16]:

```
# Centroids are found on the scaled data, so convert them back to the original units
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
centroids_df = pd.DataFrame(centroids, columns=features).round(2)
centroids_df.index.name = 'Cluster'

print("Coordinates of the cluster centroids (original units):")
centroids_df
```

Out [16]:

Coordinates of the cluster centroids (original units):

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.30	49.52
1	86.54	82.13
2	25.73	79.36
3	88.20	17.11
4	26.30	20.91

5 rows × 2 columns

In [17]:

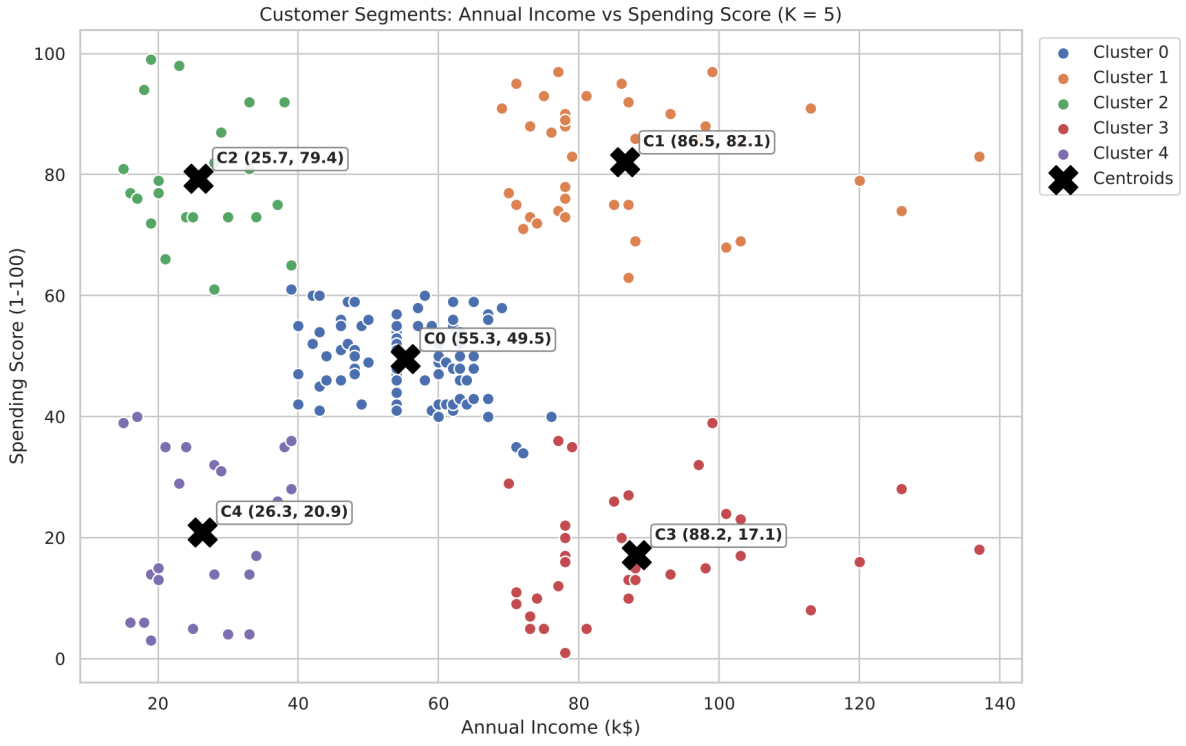
```
colors = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B3']

plt.figure(figsize=(11, 7))
for c in range(optimal_k):
    members = df[df['Cluster'] == c]
    plt.scatter(members['Annual Income (k$)'], members['Spending Score (1-100)'],
                s=60, color=colors[c], edgecolor='white', label=f'Cluster {c}')

plt.scatter(centroids[:, 0], centroids[:, 1], s=350, c='black', marker='X', label='Centroids')
for c, (x, y) in enumerate(centroids):
    plt.annotate(f'C{c} ({x:.1f}, {y:.1f})', (x, y), textcoords='offset points', xytext=(12, 10),
                 fontsize=10, fontweight='bold',
                 bbox=dict(boxstyle='round,pad=0.25', fc='white', ec='gray', alpha=0.9))

plt.title('Customer Segments: Annual Income vs Spending Score (K = 5)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(loc='upper left', bbox_to_anchor=(1.01, 1))
plt.tight_layout()
plt.show()
```

Out [17]:



> **Observation:** The five clusters are clearly separated and each centroid lies at the centre of its group. The centroids are: C0 (55.3, 49.5), C1 (86.5, 82.1), C2 (25.7, 79.4), C3 (88.2, 17.1) and C4 (26.3, 20.9), given as (annual income in k$, spending score).

Cluster Analysis

Task 9: Number of customers in each cluster

In [18]:

```
cluster_counts = df['Cluster'].value_counts().sort_index()
counts_df = pd.DataFrame({'Customers': cluster_counts,
                          'Percentage (%)': (cluster_counts / len(df) * 100).round(1)})
counts_df
```

Out [18]:

Cluster	Customers	Percentage (%)
0	81	40.5
1	39	19.5
2	22	11.0
3	35	17.5
4	23	11.5

5 rows × 2 columns

> **Observation:** Cluster 0 is the largest segment with 81 customers (40.5%), followed by cluster 1 (39), cluster 3 (35), cluster 4 (23) and cluster 2 (22).

In [19]:

```
plt.figure(figsize=(8, 4.5))
bars = plt.bar(cluster_counts.index.astype(str), cluster_counts.values, color=colors)
plt.bar_label(bars)
plt.title('Number of Customers in Each Cluster')
plt.xlabel('Cluster')
plt.ylabel('Customers')
plt.tight_layout()
plt.show()
```

Out [19]:



Task 10: Average Annual Income and Spending Score of each cluster

In [20]:

```
profile = df.groupby('Cluster').agg(
    Customers=('CustomerID', 'count'),
    Avg_Annual_Income=('Annual Income (k$)', 'mean'),
    Avg_Spending_Score=('Spending Score (1-100)', 'mean'),
    Avg_Age=('Age', 'mean'),
    Female_Percent=('Gender', lambda g: (g == 'Female').mean() * 100)).round(2)

# Name each segment from its average income and spending score
def level(value, low, high):
    return 'High' if value > high else ('Low' if value < low else 'Average')

def segment_name(row):
    income = level(row['Avg_Annual_Income'], 40, 70)
    spend = level(row['Avg_Spending_Score'], 35, 65)
    return f"{income} income, {spend} spending"

profile['Segment'] = profile.apply(segment_name, axis=1)
profile
```

Out [20]:

Cluster	Customers	Avg_Annual_Income	Avg_Spending_Score	Avg_Age	Female_Percent	Segment
0	81	55.30	49.52	42.72	59.26	Average income, Average spending
1	39	86.54	82.13	32.69	53.85	High income, High spending
2	22	25.73	79.36	25.27	59.09	Low income, High spending
3	35	88.20	17.11	41.11	45.71	High income, Low spending
4	23	26.30	20.91	45.22	60.87	Low income, Low spending

5 rows × 6 columns

> **Observation:** Each cluster has a clear profile. Cluster 1 has high income ($86.54k) and high spending (82.13) and is younger on average (32.7 years). Cluster 3 has similar income ($88.20k) but the lowest spending (17.11). Cluster 2 has low income ($25.73k) but high spending (79.36) and is the youngest group (25.3 years). Cluster 4 has low income and low spending, and cluster 0 is the average customer on both features.

Marketing Strategies for Each Segment

Cluster	Segment	Customers	Suggested marketing strategy
1	High income, high spending (**Premium / Target customers**)	39	Loyalty and VIP programmes, early access to new collections, premium brands and personalised offers to keep them spending.
3	High income, low spending (**Careful spenders**)	35	They can afford more but buy little. Use quality-focused promotions, exclusive experiences and personalised recommendations to increase their spending.
2	Low income, high spending (**Young impulsive buyers**)	22	Trendy products, flash sales, student discounts and buy-now-pay-later or reward-point schemes; avoid pushing expensive products.
0	Average income, average spending (**Standard customers**)	81	The largest group. Use regular promotions, seasonal sales, combo offers and a basic loyalty card to keep them coming back.
4	Low income, low spending (**Budget customers**)	23	Value packs, discount coupons and essential products at low prices; keep marketing costs low for this group.

Result

Thus the Mall Customer dataset was explored, the Annual Income and Spending Score features were standardised, and the Elbow Method showed that the optimal number of clusters is K = 5 (silhouette score 0.5547). The K-Means model grouped the 200 customers into five segments, the cluster labels were added to the dataset and saved as Mall_Customers_Clustered.csv, the clusters and their centroids were visualised, and the size and average income and spending score of each cluster were used to recommend a marketing strategy for every segment.